

Reproduction of Quant-dllm

czh

2026 年 7 月 6 日

演示目标

- 复现对象：Quant-dLLM 在 LLaDA-8B-Base 上的 2-bit weight-only PTQ。
- 当前主线：column-group DAQ/ABMP + DOR + GPTQ/OBC-style compensation。
- 主要入口：tools/quant_stagewise_pipeline.py 和 scripts/local_stagewise_quant_eval.sh。
- 验证目标：WinoGrande 从早期复现的 0.6267 提升到当前主线的 0.6614。

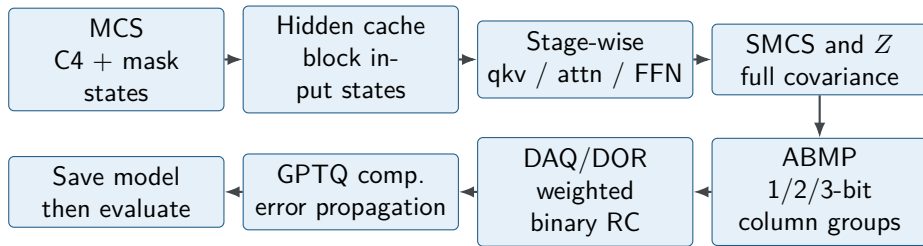
算法模块

- MCS/load_dataset.py
构造 C4 校准样本和 timestep mask states。
- utils/generate_imp_mat.py
收集 SMCS, 构造 Z , 处理 inverse diagonal。
- DAQ/quantizer.py
实现 RSR、DOR mask 和 binary pattern search。
- ABMP/allocator.py
保留早期接口; 主线分配逻辑在 main.py。

流程与文档

- tools/quant_stagewise_pipeline.py
分 block、分 stage 收集统计并写回量化权重。
- scripts/local_stagewise_quant_eval.sh
包装量化、模型保存和 LLaDA benchmark。
- docs/stagewise_gptq_daquantization.md
记录当前最对齐论文的公式、配置和结果。

复现流程



每个 stage 的统计 Z cache 来自当前已经部分量化的模型状态。

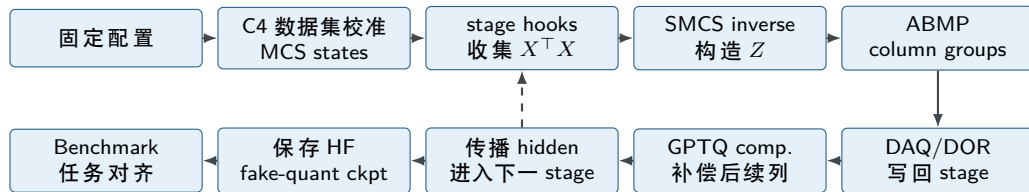
主线逻辑

Diffusion LLM 的 mask/time-step 激活分布不同于 AR LLM，因此传统 GPTQ 类 PTQ 在 W2A16/W3A16 下会明显退化；引入 binarization、MCS、DAQ/DOR 和 ABMP。

- ① MCS 构造 timestep-aware masked calibration set。
- ② 逐个 Linear 层收集输入激活，计算 importance matrix Z 。
- ③ ABMP 根据 Z 对 column block 分配 1/2/3 bit。
- ④ DAQ 用多阶 1-bit 分量叠加近似 K-bit 权重。
- ⑤ DOR 对 Z -outlier 提高重构权重。
- ⑥ 量化目标层后进入评测和消融对齐。

MCS $\rightarrow$ layer-by-layer ($Z \rightarrow$ ABMP $\rightarrow$ DAQ)

复现流程



“layer-by-layer” 进一步细化为 block 内四个 stage: qkv、attn_out、ff_in、ff_out。每个 stage 量化后立即写回，因此后续统计看到的是当前量化状态。

从复现到定位问题的实验闭环

1. 对齐配置

- 固定 nsamples=128
- 固定 seqlen=4096
- 固定 block/group=128

2. 对齐链路

- 先复测 FP baseline
- 再测
GPTQ/GPTAQ/Slim-LLM
- 最后测 Quant-dLLM
- 只用 full benchmark 做论文对比

3. 定位差距

- 看 Z -weighted error
- 看 bit allocation map
- 消融 granularity / damping

当前判断

复现主线已经把之前 0.61–0.63 的 WinoGrande 恢复到 0.6614；剩余差距更像是 FP 评测链路、任务细节和论文未公开超参共同造成，因为 FP 模型的评测精度也会比论文略低 1-2 个百分点

当前主线配置

校准与 SMCS

- nsamples=128, seqlen=4096
- num_steps=20, linear schedule
- mcs_no_full_visible=true
- mcs_normalization=tokens
- max_tokens_per_state=0
- damping_mode=diag_mean
- damp_percent=0.01

量化与分配

- daq_granularity=column
- abmp_granularity=column
- abmp_rank_scope=weight
- quant_block_size=128
- allocate_ratio=0.05
- daq_component=rsr_w_dor
- gptq_compensation=1

MCS 对齐 dLLM 的可见性分布

代码位置

MCS/load_dataset.py 负责 C4 采样、mask token 查询、timestep grid 和 Bernoulli visibility。

- 每条 C4 序列长度固定为 4096，校准样本数固定为 128。
- 前缀比例 $\gamma = 0.25$ ：前缀始终可见，后续 token 由 timestep 的 $\alpha(t)$ 控制。
- mcs_no_full_visible 去掉完全可见 endpoint，避免校准统计偏向普通 AR 输入。
- 生成的 calibration states 进入 stage-wise pipeline，后续 hooks 收集每个 Linear 的 $X^\top X$ 。

每个 transformer block 内的顺序

- ① qkv: q_proj, k_proj, v_proj
- ② attn_out: attn_out
- ③ ff_in: ff_proj, up_proj
- ④ ff_out: ff_out

- 对当前 stage 注册 hooks, 只收集该 stage 的输入统计。
- 量化后立即写回 in-memory model。
- 后续 stage 看到的是已经量化过的上游模块。
- block 完成后, 传播完整量化 block 输出作为下一层 hidden cache。

SMCS 与 Z 构造是 DOR 和 ABMP 的共同来源

输出优化目标

$$\left\| XW^\top - XQ^\top \right\|_F^2 = \text{Tr}((W - Q)S(W - Q)^\top), \quad S = X^\top X / N.$$

论文实现的 Z

$$\epsilon = 0.01 \cdot \text{mean}(\text{diag}(S)), \quad H = (S + \epsilon I)^{-1}, \quad Z_{ij} = \left(\frac{W_{ij}}{\max(\text{diag}(H)_j, \tau)} \right)^2.$$

Z 同时决定 ABMP 的 block score 和 DAQ/DOR 的 outlier mask。

分组

$$G_k = W[:, 128k : 128(k + 1)], \quad s(G_k) = \sum_{(i,j) \in G_k} Z_{ij}.$$

- `abmp_rank_scope=weight`: 每个 Linear 只和自己的 column groups 排序。
- `allocate_ratio=0.05`: 最低 5% 分到 1 bit, 最高 5% 分到 3 bit。
- 中间 groups 保持 2 bit, 使每个权重矩阵平均 bit-width 接近 2。
- column group 与 GPTQ compensation 的顺序更新天然匹配。

DAQ/DOR 把重要性压缩成可解的加权 Frobenius 目标

DOR mask

$$\Pi_{ij} = \mathbf{1}\{|Z_{ij} - \mu(Z_{\text{unit}})| > 3\sigma(Z_{\text{unit}})\}, \quad \Lambda_{ij} = 1 + (\lambda - 1)\Pi_{ij}.$$

DAQ 表示

$$Q = \sum_{t=1}^b \left(\alpha_r^{(t)} (\alpha_c^{(t)})^\top \right) \odot B^{(t)}, \quad B^{(t)} \in \{-1, +1\}^{\text{rows} \times \text{cols}}.$$

DAQ/quantizer.py 通过 closed-form scale updates 和 2^b pattern search 交替优化。

论文并没有显式提到做 GPTQ style compensation, 但缺乏该优化的 pipeline 评测效果离论文有一定差距, 通过查阅论文引用的 ARB-LLM 和 Bi-LLM, 其中均有提到工程实现用类似 GPTQ 的 block-wise compensation 进行改造, 这一优化很可能是默认的工程实现

按列组顺序修正后续残差

对 column group $c_s : c_e$ 量化后, 当前实现用 inverse Cholesky factor U 更新后续列:

$$E = (W_{\text{work}}[:, c_s : c_e] - Q[:, c_s : c_e]) / \text{diag}(U_{\text{block}}),$$

$$W_{\text{work}}[:, c_e :] = W_{\text{work}}[:, c_e :] - EU_{\text{cross}}.$$

- 该路径要求 `daq_granularity=column`。
- 它使用 full-SMCS inverse factor, 不适合旧的 blockdiag Z cache。
- 这是早期 baseline 到当前主线提升的主要实现差异。

主入口与工具脚本的分工

文件	使用场景
main.py	通用离线量化入口；支持 Z cache、ABMP、DAQ、row/tile/column ablations。
tools/quant_stagewise_pipeline.py	当前最强主线；stage-wise 统计、量化写回、GPTQ compensation。
tools/quant_layerwise_pipeline.py	layer-wise 复现路径；用于对照论文的 layer-wise 描述。
tools/build_full_smcs_z_cache.py	full-SMCS Z cache 构造；用于诊断或非主线实验。
tools/diagnose_full_smcs.py	full-SMCS 与 blockdiag Z 的一致性诊断。
tools/diagnose_quant_alignment.py	precision map、mask 覆盖率、重构误差和 Z -weighted error 诊断。

主线和消融分开运行

- `scripts/local_stagewise_quant_eval.sh`: 当前主线入口, 量化后可直接运行 benchmark。
- `scripts/exp_quantdllm_ablation_quantize.sh`: ABMP ratio、calibration size、granularity 消融。
- `scripts/exp_quantdllm_ablation_eval_mmlu.sh`: 批量评测量化模型的 MMLU。
- `scripts/exp_gptq_llada8b_baseline.sh`: 调用 QDLM/AutoGPTQ 作为 baseline。
- `scripts/exp_cross_model_template.sh`: Dream 或其他 LLaDA 变体的迁移模板。
- `llada_benchmark.sh`: 评测封装, 复用 QDLM 风格的任务和 Monte Carlo 设置。

可复现主线命令

```
RUN_ID=stagewise_gptq_comp_colabmp_n128_s4096_t20_tok0_diag1p_$(date +%Y%m%d_%H%M%S) \  
CUDA_VISIBLE_DEVICES=0,1,2 \  
PYTHON_BIN=/root/miniconda3/envs/qdlim/bin/python \  
C4_SOURCE=/root/data-fs/c4-train/c4-train.000000-of-01024.json.gz \  
MAX_TOKENS_PER_STATE=0 \  
DAMPING=0.0 DAMPING_MODE=diag_mean DAMP_PERCENT=0.01 \  
DAQ_GRANULARITY=column ABMP_GRANULARITY=column ABMP_RANK_SCOPE=weight \  
QUANT_BLOCK_SIZE=128 ALLOCATE_RATIO=0.05 DEFAULT_BITS=2 \  
DAQ_STEPS=10 IMP_LAMDA=2.0 DOR_MASK_SCOPE=unit \  
DAQ_COMPONENT=rsr_w_dor GPTQ_COMPENSATION=1 DAQ_FULLS_REFINE=0 \  
RUN_EVAL=1 EVAL_TASKS=winogrande EVAL_LIMIT=full \  
bash scripts/local_stagewise_quant_eval.sh
```

执行约束

EVAL_LIMIT=full 才能和论文表格对比； EVAL_LIMIT=200 只用于 smoke test。

输出目录

量化产物

- outputs/<run_id>/
- stagewise_args.json
- precision_map.json
- importance_scores.json
- quantized_blocks/*.safetensors
- 重写后的 HF checkpoint

诊断产物

- z_cache/block_XX.safetensors
- inverse report
- hidden cache
- benchmark results
- run logs and pid files

变体	用途
daq_fulls_refine	固定 B 后用 full-S Hessian 细化 scales; 当前不是解释 Wino-Grande 提升的必要项。
daq_activation_diag	使用 activation diagonal 作为求解权重; 属于 activation-aware DAQ 诊断。
daq_row_center / daq_block_row_center	row-centering 消融; 当前主线关闭。
tile / row / weight granularity full-SMCS Z cache	分块方式消融; 当前主线使用 column groups。 诊断 full covariance 和 blockdiag 近似差异, 不作为 stage-wise 主线缓存复用。

论文报告的 LLaDA-8B-Base 目标结果

Model	Method	MMLU(5)	Wino.(5)	PIQA(0)	ARC-C(0)	ARC-E(0)	Hella.(0)	BBH(0)	Avg
LLaDA-8B-Base	FP	65.76	74.59	74.70	44.11	74.20	54.27	42.60	61.46
	GPTQ	34.75	57.85	57.45	22.44	37.04	33.77	4.06	35.34
	GPTAQ	34.73	59.04	56.42	22.27	38.17	35.13	5.34	35.87
	Slim-LLM	47.98	60.85	62.46	26.11	53.41	39.25	6.64	42.39
	Quant-dLLM	56.87	68.19	69.75	36.26	68.69	46.45	32.18	54.06

原论文在 7 个任务上报告 Quant-dLLM 平均 54.06，其中 WinoGrande 为 68.19，BBH 为 32.18。

本次复现结果汇总

Model	Method	MMLU(5)	Wino.(5)	PIQA(0)	ARC-C(0)	ARC-E(0)	Hella.(0)	BBH(0)	Avg
LLaDA-8B-Base	FP	—	73.40	72.25	41.81	73.40	—	40.70	60.31
LLaDA-8B-Base	GPTQ	34.75	57.85	57.45	22.44	37.04	33.77	4.06	35.34
LLaDA-8B-Base	GPTAQ	34.73	59.04	56.42	22.27	38.17	35.13	5.34	35.87
LLaDA-8B-Base	Slim-LLM	47.98	60.85	62.46	26.11	53.41	39.25	6.64	42.39
LLaDA-8B-Base	Quant-dLLM	—	66.14	65.67	34.90	67.13	—	28.47	52.46

口径说明

由于时间问题，复现 Quant-dLLM 缺少 MMLU 和 HellaSwag，Avg 只统计 WinoGrande、PIQA、ARC-C、ARC-E、BBH 五个共同任务。

同任务对齐后的差距分析

Task	Paper Quant	Reproduced Quant	Gap	Local FP retention
WinoGrande	68.19	66.14	-2.05	90.1%
PIQA	69.75	65.67	-4.08	90.9%
ARC-C	36.26	34.90	-1.36	83.5%
ARC-E	68.69	67.13	-1.56	91.5%
BBH	32.18	28.47	-3.71	70.0%
Common-task Avg	55.01	52.46	-2.55	87.0%

- 绝对差距为 2.55 %，但本地 FP 在五个共同任务上也比论文 FP 低 1.73 %。
- 考虑 FP 链路差异，可以说明主线已经接近论文结果。
- 主要差异来自 PIQA 和 BBH；ARC-C 与 ARC-E 的相对对齐更好。

附录：模块依赖图

